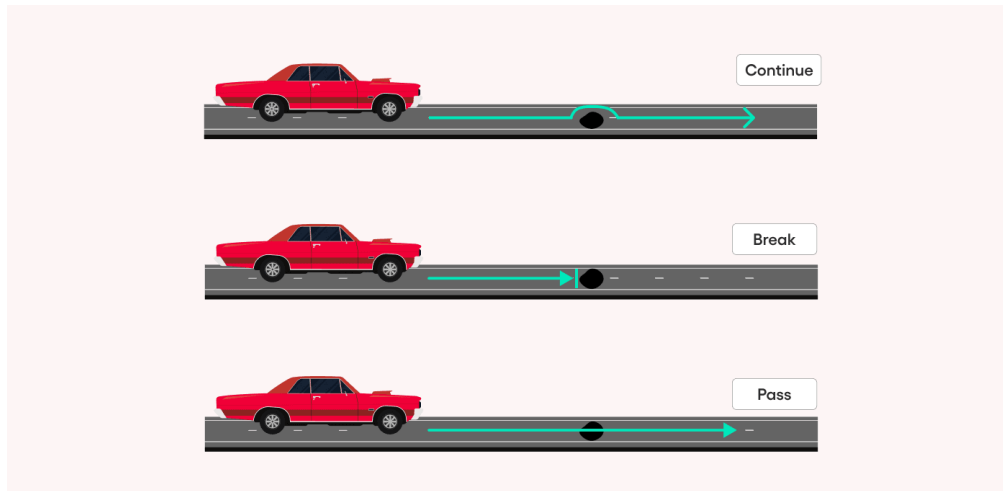


Python-Day-06

Transfer statements



In Python, transfer statements are used to change the normal flow of execution in a program. The primary transfer statements in Python include:

1. break – Exits a loop prematurely.

Used inside for and while loops to stop execution when a condition is met.

```
for i in range(5):
```

```
    if i == 3:
```

```
        break # Terminates the loop when i is 3
```

```
    print(i)
```

Output:

0

1

2

2. continue

– Skips the current iteration and moves to the next.

Used inside loops to skip a specific iteration when a condition is met.

```
for i in range(5):  
    if i == 3:  
        continue # Skips the iteration when i is 3  
    print(i)
```

Output:

```
0  
1  
2  
4
```

3. pass :- A placeholder that does nothing.

Used when a statement is syntactically required but no operation is needed.

OR

Pass Statement in Python

In Python, the pass statement is used as a placeholder for code that still needs to be implemented. It is a null operation, meaning that it does nothing. It can also be used as a placeholder for loops or conditional statements that still need to be fully defined.

Why do we Use Pass Statement?

A Pass statement allows us to have an empty loop or an empty conditional clause, which are syntactically valid but don't do anything. This is particularly useful when we need a loop or conditional statement as part of our code but want to wait to execute any code within it.

```
def function():  
    pass # Placeholder for future implementation
```

4. return – Exits a function and optionally returns a value.

Used inside functions to send a value back to the caller.

```
def add(a, b):
```

```
return a + b # Returns the sum of a and b
```

```
result = add(3, 4)
```

```
print(result)
```

Output:

7

5. `exit()` / `sys.exit()` – Terminates the program.

`exit()` is a built-in function, while `sys.exit()` requires the `sys` module.

```
import sys
```

```
print("Before exit")
```

```
sys.exit() # Terminates the program
```

```
print("After exit") # This line will not execute
```

Output:

Before exit